

KAPITEL 7: BOOTSTRAP

LERNZIELE

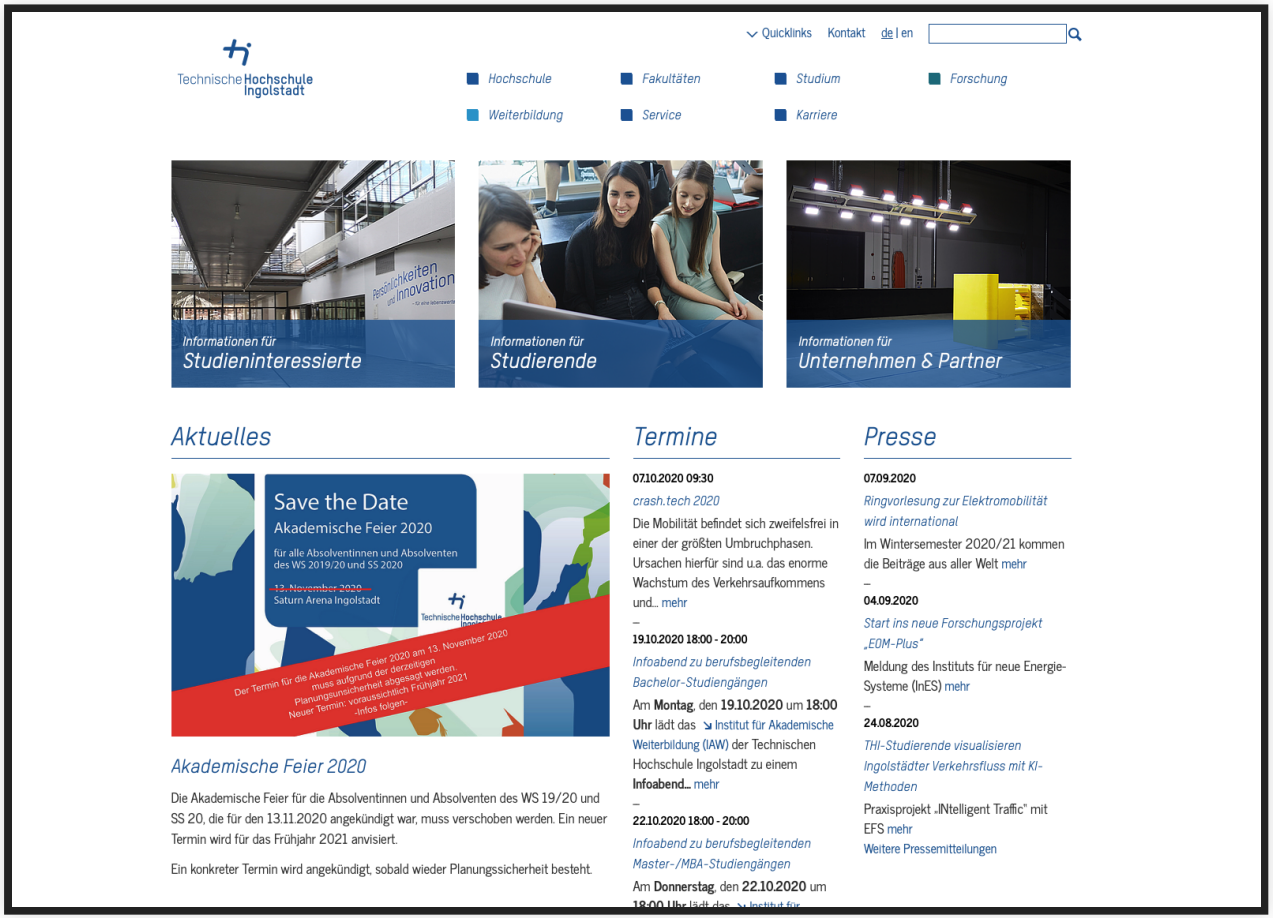
- Definieren können, was ein Frontend-(CSS-)Framework ist
- Erklären, wie sich Bootstrap im Kontext der Webtechnologien eingliedert
- Wesentliche Funktionalitäten von Bootstrap nennen
- Elemente (Grid-System, Tabellen, Cards, Tabs, Formulare und Modals) des Frameworks anwenden, um damit eine Webseite zu realisieren

7.1 MOTIVATION



Was sind grundlegende Elemente, die mit jeder Webseite realisiert werden?

Beispiel THI Webseite





Grundlegende Elemente könnten sein:

- Banner, Logo, Featurette, Carousel
- Grundlegendes Layout - Container, Spalten, Header, Footer
- Farben, Typographie
- Navigationsleisten
- Formulardesign, sowie bestimmte Stilelemente
- Responsive Webdesign

Ein Frontend-(CSS-)Framework deckt teile dieser Grundfunktionen einer Webanwendung ab und erleichtert die Erstellung individueller Lösungen.

Es gibt eine Vielzahl aktueller Bibliotheken, die einen Blick lohnen:

- W3C.CSS, Materialize, Foundation oder Pure
- Onsen UI, Ionic oder Angular Material (für mobile Webanwendungen)
- Bootstrap

BOOTSTRAP

- 2011 ursprünglich von Twitter Designer/Entwickler 2011 veröffentlicht
- aktuelle Version: Bootstrap 5
- "mobile first"-Ansatz für "responsive web apps"
- Design Templates für Formulare, Buttons, Tabellen, Navigation, Subdialoge, …



Verwendete Techniken

- HTML, CSS und JavaScript für die Komponenten
- Popper für verschiedene Funktionen in Bootstrap (früher war jQuery als Abhängigkeit notwendig, dies ist nicht mehr notwendig)
- Hier: Öffnen und Schließen von modalen Dialogen

Vertiefende Informationen

- w3schools
- Bootstrap-Dokumentation

ÜBUNG: EINSATZ VON BOOTSTRAP

Warum kann der Einsatz eines Frontend-Frameworks wie Bootstrap bei der Webentwicklung sinnvoll sein. Nennen Sie mindestens drei konkrete Vorteile.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

ÜBUNG: BOOTSTRAP-ELEMENTE

Welche grundlegenden Elemente einer Webseite können mit Bootstrap realisiert werden?

- ☐ Navigationsleisten (z.B. Drop-Down-Menüs und Hover-Effekte)
- ☐ UI-Komponenten (z.B. Modals, Carousel und Cards)
- ☐ Responsive Layouts (z.B. Grids)
- ☐ Verbesserte HTML-Generierung auf Serverseite
- ☐ Formulardesign (z.B. einheitliches Layout für Buttons und Eingaben)
- ☐ Datenbank-Verbindungen (z.B. die Anbindung an eine MySQL-Datenbank)



7.2 CONTAINER

Bootstrap definiert grundlegende Container für den Inhalt einer Webseite.

```
<div class="container">
    Dies ist Inhalt in einem Div-Element mit Container-Class.
</div>
```

Damit lassen sich Inhalte zentrieren, die Nutzung von Fensterbreiten steuern und ein grundlegendes responsives Verhalten umsetzen.

Beispiel Container

```
<div>
    Dies ist normaler Inhalt in einem Div-Element.
</div>
<div class='container'>
    Dies ist Inhalt in einem Div-Element mit Container-Class.
</div>
<div class='container-fluid'>
    Dies ist Inhalt in einem Div-Element mit Container-Fluid-Class.
</div>
```

Erläuterung

- Div-Elemente haben kein Margin, Padding und spannen über die gesamte Breite
- Container sind ein grundlegendes Layout-Element in Bootstrap
- Klasse `container-fluid` erlaubt das Nutzen der verfügbaren Breite für den Inhalt
- Klasse `container` erlaubt das Zentrieren von Inhalt
- Schachtelungen möglich
- Container-Klassen (u.a. `container`) reagieren responsiv

Responsive Breakpoints

Kategorie	Verfügbare Breite	Media-Query
Smartphones	`576px <= breite < 768px`	`@media(min-width: 576px) {…}`
Tablets	`768px <= breite < 992px`	`@media(min-width: 768px) {…}`
Desktops	`992px <= breite < 1200px`	`@media(min-width: 992px) {…}`
Große Desktops	`1200px <= breite`	`@media(min-width: 1200px) {…}`

	Extra small	Small	Medium	Large	X-Large	XX-Large
Break-Point	*<576px*	*≥576px*	*≥768px*	*≥992px*	*≥1200px*	*≥1400px*
.container	100%	540px	720px	960px	1140px	1320px
.container-sm	100%	540px	720px	960px	1140px	1320px
.container-md	100%	100%	720px	960px	1140px	1320px
.container-lg	100%	100%	100%	960px	1140px	1320px
.container-xl	100%	100%	100%	100%	1140px	1320px
.container-xxl	100%	100%	100%	100%	100%	1320px
.container-fluid	100%	100%	100%	100%	100%	100%

ÜBUNG: CONTAINER-KLASSEN

Welche Container-Klasse ist zu verwenden, wenn der Inhalt stets die volle Breite des Viewports einnehmen soll?

- [ ] `.container-fluid`
- [ ] `.container-lg`
- [ ] `.container-sm`
- [ ] `.container`

ÜBUNG: CONTAINER-VERWENDUNG

Welche Aussagen über Bootstrap Container sind korrekt?

- [ ] Container können ausschließlich für das Grid-System verwendet werden
- [ ] Der `.container` verwendet bei allen Breakpoints die volle Bildschirmbreite
- [ ] `.container-fluid` nutzt immer die volle verfügbare Breite des Viewports
- [ ] Ein `.container` zentriert seinen Inhalt automatisch horizontal

ÜBUNG: CONTAINER-LAYOUT IN CSS

Erläutern Sie, wie das folgende Bootstrap Container-Layout mit nativen CSS-Eigenschaften umgesetzt werden könnte.

```
<div class="container">
    <div>Inhalt</div>
</div>
```

Hinweis: Untersuchen Sie die Bootstrap-Container mit den Entwicklungstools im Browser.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt



7.3 GRID

Grid-System zur Realisierung eines Spaltenlayouts mit 12 Spalten.

```
<div class="container-fluid">
    <div class="row">
        <div class="col-sm">first</div>
        <div class="col-sm">second</div>
        <div class="col-sm">third</div>
    </div>
</div>
```

Verwendung

- Das Grid hat 12 Spalten
- Jede Zeile muss durch `<div class="row">` beginnen
- Spalten werden mittels `col-<display-typ>-<anzahl-abschnitte>` angegeben
- Beispiel `<div class="col-sm-4">` bedeutet, dass über 4 von 12 Abschnitten gespannt wird und die Ansicht für kleine Displays geeignet ist. Ist die Ansicht kleiner, werden die Spalten zeilenweise dargestellt.

Grundzüge des Grid-Systems:

- Technisch basiert es auf dem Konzept der Flexbox (s. https://css-tricks.com/snippets/css/a-guide-to-flexbox)
- Ein Container wird in mehrere Zeilen (rows) eingeteilt, hierfür gibt es die Klasse `row`
- Eine Row wird in mehrere Spalten aufgeteilt. Jede Spalte besteht aus 1-12 Abschnitten, wobei die Summe der Abschnitte genau 12 ergeben muss.
- Ein Abschnitt ist ein vertikaler Streifen in der Row. Alle 12 Abschnitte einer Row teilen sich die verfügbare Breite der Row zu gleichen Teilen auf.

ÜBUNG: GRID-LAYOUT IN CSS

Erläutern Sie kurz, wie das folgende Grid-Layout mit CSS-Eigenschaften (Block / Inline-Block und Media-Queries) umgesetzt werden könnte.

```
<div class="row">
    <div class="col-sm-4">Spalte 1</div>
    <div class="col-sm-4">Spalte 2</div>
    <div class="col-sm-4">Spalte 3</div>
</div>
```

Hinweis: Untersuchen Sie die Bootstrap-Container mit den Entwicklungstools im Browser.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

7.4 TABELLEN

Grundlegende Tabelleneigenschaften (Border, Hover, Padding, usw.)

```
<div class="container-fluid">
    <table class="table table-hover table-striped">
        <!-- ... -->
    </table>
</div>
```

Erläuterungen zu einigen Tabellenklassen

- `table-sm` (bzw. table-md, table-lg etc.): passt die Spaltenbreiten an ein kleines (mittleres, großes etc.) Display an
- `table-dark`: Dunkle Variante einer Tabelle
- `table-hover`: fährt die Maus über eine Zeile, wird diese grau hinterlegt
- `table-bordered`: fügt jeder Tabellenzelle einen Rand hinzu
- `table-borderless`: entfernt alle Ränder
- `table-striped`: jede ungerade Zeile wird grau (#dee2e6) hinterlegt

Responsive Table

Wie reagieren Tabellen, wenn der Platz nicht mehr ausreicht?

Mit einem Div-Container um eine Tabelle, welcher die Klasse `table-responsive` verwendet, kann gesteuert werden, dass ein horizontaler Scroll-Balken entsteht, wenn die Seite zu wenig platz bietet.

7.5 BUTTONS



Verschiedene Button-Stile: Primär, Sekundär, Achtung, Warnung...

```
<button type="button" class="btn btn-primary">Basic Button</button>
<button type="button" class="btn btn-secondary">Basic Button</button>
```

Erläuterung zum Beispiel

- Es werden die drei Eigenschaften Stil, Größe und Aktiv/Inaktiv vorgestellt
- Mithilfe der Stile wird eine Grundmenge an einheitlichen Buttonstilen definiert
- Mithilfe der Größe lässt sich in seiner Breite und Schriftgröße verändern
- Mithilfe der Klassen für Aktiv und Inaktiv kann gesteuert werden, ob der Button gedrückt werden kann

ÜBUNG: BOOTSTRAP BUTTONS

Welche CSS-Eigenschaften sind grundlegend für die Gestaltung eines Buttons im Bootstrap-Stil?

- [ ] Padding und Line-Height für eine angemessene Button-Größe
- [ ] Eine Hintergrundfarbe und eine kontrastierende Textfarbe sowie deren Hover-Zustände
- [ ] `position: absolute` für die korrekte Platzierung im Layout
- [ ] Border-Radius für abgerundete Ecken und eine passende Border
- [ ] Eine einheitliche Breite für alle Button-Varianten



7.6 CARDS

Cards haben einen Header, Body und Footer

```
<div class="card">
    <div class="card-header">Test</div>
    <div class="card-body">
        <p class="card-text">Lorem Ipsum...</p>
    </div>
    <div class="card-footer">Footer...</div>
</div>
```



7.7 TABS

Grundelement Tabs, vergleichbar mit Browser-Tabs, Dateibrowser-Tabs oder Konfigurations-Tabs.

```
<div class="container">
    <ul class="nav nav-tabs">
        <li class="nav-item">
            <a class="nav-link active" data-toggle="tab" href="#t1">Tab 1</a
        </li>
    </ul>
</div>
<div class="tab-content">
    <div class="tab-pane container active" id="t1">
        <p class="tab-inhalt">
            Tab 1 Inhalt
        </p>
    </div>
</div>
```



Erläuterung

- Jeder Tab muss mit einem entsprechenden Inhalt verbunden werden
- Entscheidend für die Steuerung ist das Attribut `data-toggle` und die Referenz auf das entsprechende Inhaltselement über `href`
- Referenziert wird über die ID eines Elementes und dem Fragment einer URL (beginnend mit einem `#`)
- Die Angabe im Link `<a data-toggle="tab" href="#tab1">...</a>` bezieht sich somit auf `<div class="tab-pane" id="tab1">`
- Die Klasse `fade` erlaubt eine Überblende
- Mit der Klasse `active` wird der erste aktive Tab angegeben



7.8 FORMULARE

Input-, Textarea und Schalter-Elemente lassen sich in einem modernen Layout verwenden.

```
<div class="container">
    <form>
        <div class="form-group">
            <label for="text">Text</label>
            <input class="form-control" placeholder="Mail" id="text">
        </div>
        <button class="btn btn-primary">Senden</button>
    </form>
</div>
```



Erläuterung zu Form-Inputs:

- input-Tags `<input type="…" …>`: hier gibt es Styles für `readonly`, `disabled` und die Darstellung als statischer Text (plaintext)
- Einfach- oder Mehrfachauswahl mit `select`
- Mehrzeilige Textfelder mit `<textarea>`
- Check- und Radio-Boxen: `form-check` als Klammer, um das Label (`form-check-label`) und die Check- oder Radiobox (`form-check-input`)
- Das Suffix `-inline` stellt alle Inputs in einer Zeile dar (z.B. “verschlüsseln” / “signieren”)

VALIDIERUNG

Bootstrap bietet über die Formularkomponenten Möglichkeiten zur Visualisierung von validen und nicht validen Eingaben.

Toggle CSS-Klassen

Eine Möglichkeit ist die Verwendung der Klassen is-valid und is-invalid, und bei einer Überprüfung diese entsprechend zu aktivieren bzw. deaktivieren.



Toggle Custom Validity

Das HTML-Attribut erlaubt es an einem Input-Element die Eingabe als Valide oder Invalide zu markieren, indem mittels `setCustomValidity()` ein Wert gesetzt wird. Wichtig ist, dass das Formular die Klasse `was-validated` erhält.

ÜBUNG: FORMULAR-LAYOUT

Wie kann ein zweispaltiges Formular (links Labels, rechts Eingabefelder) responsiv mit Bootstrap umgesetzt werden?

- [ ] Man kann `float: left` und `float: right` für Label und Input verwenden
- [ ] Mit dem Grid-System: Labels in `col-sm-3`, Eingabefelder in `col-sm-9` innerhalb einer `row`
- [ ] Für responsive Darstellung ist ein zusätzliches @media-Query erforderlich
- [ ] Die Summe der Grid-Spalten (3+9) ergibt die für Bootstrap benötigten 12 Spalten

ÜBUNG: FORMULAR-VALIDIERUNG

Welche Aussagen zur Validierung von Formularen mit Bootstrap sind korrekt?

- [ ] Die Klassen `is-valid` und `is-invalid` werden von Bootstrap automatisch gesetzt
- [ ] Die JavaScript-Methode `setCustomValidity()` kann in Kombination mit der Klasse `.was-validated` genutzt werden
- [ ] Die Validierung erfolgt automatisch ohne JavaScript
- [ ] Mit den Klassen `is-valid` und `is-invalid` können Eingabefelder als valide oder invalide markiert werden

7.9 MODAL

Ein Dialog, der sich über der Seite öffnet und überlagerte Elemente sperrt

```
<div class="modal" id="dialog">
    <div class="modal-dialog">
        <div class="modal-content">
            <div class="modal-header">
                <h4 class="modal-title">Titel</h4>
                <button type="button" class="btn-close"
                    data-bs-dismiss="modal"></button>
            </div>
            <div class="modal-body">
                <div class="container">
                    <p>Inhalt</p>
                </div>
            </div>
            <div class="modal-footer">
                <button type="button" class="btn btn-primary btn-block"
                    onclick="login()">Login (im footer)</button>
            </div>
        </div>
```

Als Grundelemente sind mindestens `modal` und `modal-dialog` notwendig.
Weitere Elemente konfigurieren das Modal im Detail.

Erläuterung zu Modal

- Ein **Modal** ist ein Dialog, der sich über der Seite öffnet und überlagerte Elemente sperrt, solange das „modal“ sichtbar ist.
- Wichtig: ID des Modals (hier: modalId)
- Öffnen eines Modals:
  - Per Button (oder Link): `<button … data-bs-toggle="modal" data-bs-target="#dialog">`
  - JavaScript:

```
let myModalEl = document.getElementById('dialog')
let modal = bootstrap.Modal.getOrCreateInstance(myModalEl)
modal.show(); // bzw. modal.hide();
```

- Schließen eines Modals:
  - Per Button (oder Link): `<button … data-dismiss="modal">`
  - JavaScript/jQuery:

```
let myModalEl = document.getElementById('dialog')
let modal = bootstrap.Modal.getOrCreateInstance(myModalEl)
modal.hide();
```

- Grundstruktur `<div class="modal">`, darin `<div class="modal-dialog">`
  - Kopfbereich, Klasse `modal-header`
  - Inhalt: Klasse `modal-body`
  - Fußbereich: Klasse `modal-footer`

ÜBUNG: ÖFFNEN VON MODALS

Welche Aussagen über Bootstrap Modals sind korrekt?

- ☐ Ein Modal kann sowohl über `data-bs-toggle="modal"` als auch über die JavaScript-Methode `modal()` gesteuert werden
- ☐ Die Grundstruktur eines Modals erfordert mindestens die Klassen `.modal` und `.modal-dialog` für korrektes Verhalten
- ☐ Die Klasse `.modal-content` ist optional und nur für die Gestaltung
- ☐ Modals können verschachtelt werden

ÜBUNG: SCHLIESSEN VON MODALS

Welches JavaScript-Statement ist korrekt, um ein Modal in Bootstrap 5 programmatisch zu schließen?

- [ ] `document.getElementById('dialog').hide();`
- [ ] `$('#dialog').modal('hide');`
- [ ] `bootstrap.Modal.hide('#dialog');`
- [ ] `const modal = bootstrap.Modal.getOrCreateInstance('#dialog');modal.hide();`

ÜBUNG: FUNKTIONSWEISE

Erklären Sie kurz, welche Komponenten notwendig wären, um ein Modal-Fenster mit HTML, CSS und JavaScript nachzubauen (ohne Bootstrap).

Hinweis: Untersuchen Sie das Grid-Layout mit den Entwicklungstools im Browser.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

7.10 ÜBUNGEN

STUDENTENTABELLE

Für eine Tabelle mit Studenten für den Kurs soll Bootstrap verwendet werden und die entsprechende Tabelle in HTML realisiert werden.

Nutzen Sie das bereitgestellte Projekt-Archiv bootstrap-table.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



- Integrieren Sie Bootstrap
- Ergänzen Sie eine Tabelle
- Fügen Sie die im folgenden Dargestellten Inhalte hinzu
- Sorgen Sie dafür, dass die Tabelle responsive ist, jede Zeile abwechselnd gefärbt ist und die Farbe wechselt, wenn die Maus über der Zeile ist
- Erweiterung: Ändern Sie den Farbwechsel in `lightpink` und `lightyellow` und den Effekt, wenn die Maus über der Zeile ist in `white`

Folgenden Inhalt soll die Tabelle haben:

Nr	Matrikelnummer	Nachname	Vorname	Hobbies	Slogan
1	47110001	Maier	Anna	Tanzen, Angeln	Lorem ipsum...
2	47110002	Müller	Peter	Malen, Zeichnen	keine
3	47110003	Schmidt	Paula	keine	Hakuna Matata
4	47110004	Max	Musterman	keine	Don't Panic
5	47110005	Erika	Musterfrau	keine	keine

PERSONEN KARTEN

Erstellen Sie eine Darstellung von Personeninformationen unter Verwendung von Cards in einem Card-Deck. Jede Card soll anschließend bearbeitet werden können.

Nutzen Sie das bereitgestellte Projekt-Archiv bootstrap-form.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



Die Karten

- Erstellen Sie ein Card-Deck
- Erstellen Sie drei Cards
    - im Header soll der Name stehen
    - im Body soll mittels Grid-Layout zu 4/12 "Wo:" und zu 8/12 der entsprechende Wert stehen
    - im Body soll mittels Grid-Layout zu 4/12 "Alter:" und zu 8/12 der entsprechende Wert stehen
    - im Footer soll ein Button mit der Aufschrift "Bearbeiten" platziert werden
- Die Div-Elemente, welche die entsprechenden Informationen enthalten (Name, Wo und Alter) sollen ein ID-Attribut mit dem Präfix der Karte erhalten: z.B. `card1-name`, `card1-wo`, usw.

Folgenden Inhalt sollen die Karten haben:

Name	Wo	Alter
Paul	Ingolstadt	21
Anna	Ingolstadt	19
Sabine	Ingolstadt	33



Das Formular

- Teilen Sie den verfügbaren Bereich mit dem Grid-System zu einem Teil mit 66,6% und einem Teil mit 33,3% auf
- Erstellen Sie in der ersten Spalte (66,6%) ein Formular unter Verwendung eines weiteren Grids (`form-group` in Kombination mit `row`) für die drei Felder Name, Wo und Alter
- Erstellen Sie in der zweiten Spalte (33,3%) zwei Buttons, einen zum `Abbrechen` und einen zum `Speichern`



Die Logik

- Ergänzen Sie auf jedem Button der Karten ein `onclick` und rufen Sie die Funktion `kontaktBearbeiten` auf - jeweils mit `card1`, `card2` oder `card3`
- Ergänzen Sie auf dem Abbrechen-Button des Formulars ein `onclick` und rufen Sie die Funktion `schliessen` auf
- Ergänzen Sie auf dem Speichern-Button des Formulars ein `onclick` und rufen Sie die Funktion `speichern` auf

PERSONEN KARTEN UND MODAL

Verschieben Sie das Formular aus der vorangegangenen Übung in ein Modal und steuern Sie dieses entsprechend.

Nutzen Sie das bereitgestellte Projekt-Archiv bootstrap-modal.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.



Anpassungen

- Im ersten Schritt soll das Formular in ein entsprechendes Modal verschoben werden
    - Achten Sie auf einen Titel
    - Das Formular soll in den Body
    - Platzieren Sie die Buttons im Footer
- Ergänzen Sie die Modalsteuerung in den JavaScript-Funktionen `kontaktBearbeiten` und `schliessen`

